Factory Method Pattern: Car Manufacturing System

For this task I used the Factory Method Pattern to solve the core problem of the brief, which is that the main program should never need to know the exact class of car it is creating. The design centres on two abstractions, Car and CarFactory, with the concrete classes sitting behind them where the client code cannot see or depend on them directly.

The Car abstract class defines a single method, drive(), which every concrete car type has to implement in its own way. Sedan, SUV and Hatchback each provide their own version of drive(), returning a description that reflects what actually makes that car type different to drive, rather than just printing a generic message. This keeps the behaviour meaningful rather than just satisfying the interface for the sake of it.

On the creation side, CarFactory is an abstract class with a factory method, create_car(), that each concrete factory overrides to return its specific car type. I also added a small deliver_car() helper on the base CarFactory class, which calls create_car() and then drive() on the result. This is the part that actually demonstrates why the pattern is useful, the client only ever calls deliver_car() on whichever factory it has been given, and never has to write Sedan(), SUV(), or Hatchback() anywhere itself.

The demonstration in the main block builds a list of the three concrete factories and loops over them, calling deliver_car() on each. Nothing in that loop knows or cares which car type it is dealing with, which is exactly the point of the pattern, adding a new car type later would only mean adding a new Car subclass and a matching factory, with no existing code needing to change.

Code

```
from abc import ABC, abstractmethod


# ----------------------------------------------------------------------
# Car interface (abstract class)
# ----------------------------------------------------------------------
class Car(ABC):
    @abstractmethod
    def drive(self):
        pass


# ----------------------------------------------------------------------
# Concrete car classes
# ----------------------------------------------------------------------
class Sedan(Car):
    def drive(self):
        return "Driving a Sedan: smooth and fuel efficient, ideal for daily
commuting."


class SUV(Car):
    def drive(self):
```

```
        return "Driving an SUV: higher ground clearance, built for rougher
terrain."


class Hatchback(Car):
    def drive(self):
        return "Driving a Hatchback: compact and easy to manoeuvre in the
city."


# ------------------------------------------------------------------------
# CarFactory abstract class
# ------------------------------------------------------------------------
class CarFactory(ABC):
    @abstractmethod
    def create_car(self):
        pass

    def deliver_car(self):
        car = self.create_car()
        return car.drive()


# ------------------------------------------------------------------------
# Concrete factories
# ------------------------------------------------------------------------
class SedanFactory(CarFactory):
    def create_car(self):
        return Sedan()


class SUVFactory(CarFactory):
    def create_car(self):
        return SUV()


class HatchbackFactory(CarFactory):
    def create_car(self):
        return Hatchback()


# ------------------------------------------------------------------------
# Demonstration
# ------------------------------------------------------------------------
if __name__ == "__main__":
    factories = [SedanFactory(), SUVFactory(), HatchbackFactory()]

    for factory in factories:
        print(factory.deliver_car())
```

Program Output

Driving a Sedan: smooth and fuel efficient, ideal for daily commuting.
Driving an SUV: higher ground clearance, built for rougher terrain.
Driving a Hatchback: compact and easy to manoeuvre in the city.

Development Notes

This solution was developed in Visual Studio Code, and the code was written and tested locally before being written up here.

AI assistance was used during this work, specifically to help review and assess the quality of the code and its adherence to the Factory Method Pattern, alongside proofreading the accompanying written explanation.